

Agentic AI

Memory & Session Management with Google ADK

 We'll build ONE running example together — SupportBot — adding one capability per topic, all the way to a working multi-user ADK agent.

ADK Sessions · Context Windows · Scratchpads · Summarization · JSON Sessions · Concurrency · State Persistence

Agenda

Every concept below adds one more piece to a single running project — SupportBot — so by the end, nothing here is theoretical.

1

Short-Term Memory — Steps 1-5

ADK session management · Context window management · Scratchpad pattern · Summarization · JSON for fast session memory

2

Managing Concurrent Sessions — Steps 6-7

Many users at once · State persistence across turns · Then: SupportBot assembled + practical project

What Is Short-Term Memory?

Short-term memory lets an agent keep track of the current conversation or task — everything it needs "in mind" right now, but not forever.

 Meet the project: over the next few slides we'll build SupportBot — a customer-support agent in ADK — one capability at a time.

✗ Without Memory

- Every turn is treated as brand new
- User must repeat their order number every message
- Can't track multi-step tasks

✓ With Short-Term Memory

- Retains recent conversation turns
- References what was said earlier
- Keeps task state while working

Session

One conversation thread — its events are the history.

State

A key/value notepad attached to that session.

Memory

Long-term, across sessions — next module.

ADK Session Management

LLM APIs are stateless. In ADK, a **Session** stores the conversation as events, and the **Runner** resends them on every turn.

session.events

user	"My order hasn't arrived."
support_bot	"Sorry to hear that — order #?"
user	"#48213"
tool	get_order_status → "shipped"

SUPPORTBOT.PY — PIECE 1 OF 7

```
support_bot = LlmAgent(name="support_bot", model=MODEL,
                        instruction="You are a support agent.",
                        tools=[get_order_status])

session_service = InMemorySessionService()
runner = Runner(agent=support_bot, app_name="supportbot",
                session_service=session_service)

session = await session_service.create_session(
    app_name="supportbot", user_id="usr_882")
```

The Runner loads the session, sends its events to the model, and appends the new ones — that IS SupportBot's conversation history.

Context Window Management

The context window is fixed. As SupportBot's conversation grows, something has to give.

System	Older History (dropped)	Recent History (kept)	Current Msg	Response
--------	-------------------------	-----------------------	-------------	----------

Truncation

Drop oldest messages.

Sliding Window

Keep only the last N turns.

Priority Retention

Keep system + recent + pinned.

Token Budgeting

Reserve tokens per section.

SUPPORTBOT.PY — PIECE 2 OF 7

```
MAX_TURNS = 20    # SupportBot picks Sliding Window

def trim_history(callback_context, llm_request):
    llm_request.contents = llm_request.contents[-MAX_TURNS:]
    return None     # None = send the trimmed request

support_bot = LlmAgent(..., before_model_callback=trim_history)
# swap for real token counts in production
```

⚠ It's NOT memory — it's a window. Once a message falls outside it, the model has zero access to it.

Scratchpad Pattern for Intermediate Reasoning

A scratchpad is where SupportBot writes its plan before replying. In ADK, a planner agent saves it to **session.state** with **output_key**.

✗ Without a Scratchpad

- Reasoning mixed into the reply
- Hard to debug a mistake
- No place for a multi-step plan

✓ With a Scratchpad

- Plan written separately
- Stored in state, apart from the reply
- Hidden, or shown as a trace

SUPPORTBOT.PY — PIECE 3 OF 7

```
planner = LlmAgent(name="planner", model=MODEL,
    instruction="List the steps to solve the issue. Don't reply.",
    output_key="scratchpad")          # saved to session.state

answerer = LlmAgent(name="answerer", model=MODEL,
    instruction="Follow this plan: {scratchpad}",
    tools=[get_order_status],
    before_model_callback=trim_history) # from step 2

support_bot = SequentialAgent(name="support_bot",
    sub_agents=[planner, answerer])

# state["scratchpad"] (hidden from the user):
# 1. user reports order #48213 missing
# 2. call get_order_status("48213")
# 3. reply with status + delivery date
```

Summarization Memory for Long Conversations

When a SupportBot session outgrows the window, ADK compresses older events into a summary instead of simply dropping them.



SUPPORTBOT.PY — PIECE 4 OF 7

```
app = App(name="supportbot", root_agent=support_bot,
          events_compaction_config=EventsCompactionConfig(
              compaction_interval=5,    # summarize every 5 turns
              overlap_size=1))         # keep 1 turn for continuity

runner = Runner(app=app, session_service=session_service)

# 500+ tokens -> "failed payment on #48213; check decline code"
```

✓ Gains

- Saves a large number of tokens
- Preserves key facts

✗ Costs

- Loses exact earlier wording
- Costs an extra model call

Simple JSON for Fast Session Memory

InMemorySessionService forgets everything on restart. A JSON file per session is the quickest, dependency-free fix — read it in, write it back out.

Session Storage

State saved to
sessions/{id}.json

Simplicity

No server — just read/write a
file

TTL / Expiry

Check a saved timestamp,
drop if too old

Human-Readable

Open the file to inspect state

Scratchpad Cache

state["scratchpad"] saved
with the rest

Portability

Back up or move a session by
copying it

SUPPORTBOT.PY — PIECE 5 OF 7

```
def save_json(session):
    path = pathlib.Path(f"sessions/{session.id}.json")
    path.write_text(json.dumps(session.state))

def load_json(session_id):
    path = pathlib.Path(f"sessions/{session_id}.json")
    return json.loads(path.read_text()) if path.exists() else {}

session = await session_service.create_session(
    app_name="supportbot", user_id="usr_882",
    session_id="s_882", state=load_json("s_882"))
```

Session Management — From One Chat to Many

Picking up where we left off: SupportBot already remembers one conversation. Now let's make it serve many users at once.

One conversation runs in one session. Many users mean many sessions running at the same time.

Session Identity

app_name + user_id + session_id
identify exactly one conversation.

Isolation

One user's state can never leak into
another user's session.

Concurrency

Many sessions run in parallel through
one async Runner.

Lifecycle

create_session · get_session ·
list_sessions · delete_session

Example: "User A asks about their order. User B asks about billing — at the exact same time. Each needs its own isolated memory."

Managing Multiple Concurrent User Sessions

Serving thousands of users at once means SupportBot's session layer has to hold up under real concurrency.

⚠ Challenges

- Race conditions on shared state
- Memory growth across sessions
- Routing a message to the right session

✓ Solutions

- One session per user_id + session_id
- Write state via tool_context, never globals
- Delete or expire idle sessions

SUPPORTBOT.PY — PIECE 6 OF 7

```
async def chat(user_id, session_id, text):
    msg = types.Content(role="user", parts=[types.Part(text=text)])
    async for event in runner.run_async(
        user_id=user_id, session_id=session_id, new_message=msg):
        if event.is_final_response():
            return event.content.parts[0].text

await asyncio.gather(
    chat("usr_882", "s_882", "Where is order #48213?"), # User A
    chat("usr_910", "s_910", "Why was I charged twice?")) # User B

# one Runner, two isolated sessions, zero shared state
```

State Persistence Across Agent Turns

For SupportBot to feel continuous, state must survive between turns — not just the chat text. A key's prefix sets how long it lives.

no prefix

This session only

user:

Every session of this user

app:

Every user of the app

temp:

This turn only, never saved

SUPPORTBOT.PY — PIECE 7 OF 7 (THE LAST PIECE)

```
session_service = DatabaseSessionService(db_url="sqlite:///supportbot.db")

def get_order_status(order_id: str, tool_context: ToolContext) -> dict:
    result = {"order_id": order_id, "status": "shipped"}
    tool_context.state["last_tool_result"] = result    # this session
    tool_context.state["user:last_order"] = order_id  # all of this user's sessions
    return result
```

🔑 Swap InMemorySessionService → DatabaseSessionService: sessions, events and state now survive a restart. Nothing else changes.

SupportBot — All 7 Pieces, Assembled

SUPPORTBOT.PY — THE COMPLETE FILE

```

session_service = DatabaseSessionService(db_url="sqlite:///supportbot.db")
def trim_history(callback_context, llm_request):          # Step 2
    llm_request.contents = llm_request.contents[-MAX_TURNS:]
planner = LlmAgent(name="planner", output_key="scratchpad") # Step 3
answerer = LlmAgent(name="answerer", tools=[get_order_status],
                    instruction="Follow this plan: {scratchpad}",
                    before_model_callback=trim_history)
support_bot = SequentialAgent(name="support_bot",
                              sub_agents=[planner, answerer])
app = App(name="supportbot", root_agent=support_bot,      # Step 4
          events_compaction_config=EventsCompactionConfig(compaction_interval=5))
runner = Runner(app=app, session_service=session_service) # Steps 1 + 7
state = load_json("s_882")                                # Step 5
await asyncio.gather(chat("usr_882", "s_882", q1),        # Step 6
                    chat("usr_910", "s_910", q2))

```

What SupportBot can now do:

- ✓ ADK sessions
- ✓ Context window management
- ✓ Scratchpad reasoning
- ✓ Summarization memory
- ✓ JSON fast memory
- ✓ Concurrent sessions
- ✓ State persistence

Your Turn: Build a Stateful Multi-Turn Agent

You just built SupportBot's memory and session system, piece by piece. Now use the same building blocks to build your own ADK agent — for any scenario you like.

Project Requirements

- A Runner + SessionService, with one session per user_id + session_id
- Full multi-turn conversation kept in session events
- Context window handling with before_model_callback (truncation or sliding window)
- A scratchpad step: a planner agent writing to state via output_key
- Summarization with EventsCompactionConfig once sessions grow
- At least one tool that saves its result with tool_context.state
- State saved to JSON first, then DatabaseSessionService
- TTL-style expiry: ignore or archive sessions past a saved timestamp

Project Architecture & Deliverables

The same three layers we just built into SupportBot — reused for your own agent.

Session Layer

- SessionService per app
- One session per user_id + session_id
- JSON → Database persistence

Memory Layer

- Events = conversation history
- Trim + summarize as it grows
- Scratchpad in state

Agent Loop

- runner.run_async each turn
- Tools write tool_context.state
- Planner → answerer flow

Deliverables

- Working agent code (repo or script)
- Short README explaining the architecture
- Live demo with 2 concurrent sessions

Evaluation Rubric

- Correctness — isolated sessions, persisted state
- Memory design — trimming + summarization used well
- Code clarity and demo quality

Ship an agent that remembers, reasons, uses a tool, and survives a restart — with no lost state.

Module Complete — What You Now Know:

- ✓ What short-term memory is, and how ADK models it as Session, State and Memory
- ✓ How the Runner and SessionService keep conversation history
- ✓ How to trim the context window with `before_model_callback`
- ✓ How a planner agent + `output_key` gives a scratchpad
- ✓ How events compaction summarizes long conversations
- ✓ How a JSON file can store and restore session state
- ✓ How one Runner serves many isolated, concurrent sessions
- ✓ How state prefixes and DatabaseSessionService persist state across turns
- ✓ How all seven pieces combined into one working agent: SupportBot

Coming up next: Long-term memory

SMIT Peshawar · Agentic AI | 96 Hours to change your career.